

TypeScript: Build a Typed To-Do App

Lesson Handout & Study Guide — Mastering Safe & Typed Code

1. Introduction: Why Do We Need TypeScript?

Imagine going shopping at a large supermarket (like D-Mart) with a messy, unorganized grocery list. If you write items down without clarity, you might mistake salt for sugar or buy the wrong quantities!

The Spice Box Analogy

TypeScript is like having clearly labeled spice jars in your kitchen. Every box tells you exactly what is inside so you never accidentally add salt instead of sugar to your tea!

What is TypeScript?

TypeScript = JavaScript + Safety Rules

JavaScript is flexible, but it allows sneaky mistakes to go unnoticed until your application crashes. TypeScript acts as your smart safety guard and proofreader compiler—catching bugs while you write your code before it ever runs!

2. Meet the Basic Data Types

In TypeScript, we use type annotations (`: type`) to give variables explicit labels:

Type	Description	Example Value	TypeScript Syntax
<code>string</code>	Text / Words enclosed in quotes	"Buy Amul Milk"	<code>let taskName: string;</code>
<code>number</code>	Numeric values (integers or decimals)	2 (Priority level)	<code>let priority: number;</code>
<code>boolean</code>	True or False conditions	<code>true / false</code>	<code>let isDone: boolean;</code>

3. Catching Sneaky Bugs Early!

When you assign a value of the wrong type to a variable, TypeScript immediately flags a **Type Error**.

```
let taskName: string = "Do Homework";

// ❌ Sneaky Bug Attempt:
taskName = 100; // Type Error: Type 'number' is not assignable to type 'string'.
```

Why did this happen? TypeScript knows that `taskName` was defined to hold text (`string`). Trying to assign a number (`100`) causes the TypeScript compiler to stop you right away, saving you from runtime bugs!

4. Designing Blueprints with Interfaces

An **Interface** in TypeScript is like a blueprint or a School ID Card. Just as a school ID card enforces that every student must have a Name, Grade, and Photo, an interface defines the exact shape and properties that an object must contain.

```
interface TodoItem {  
  title: string;  
  priority: number;  
  isDone: boolean;  
}
```

Building Real To-Do Tasks

```
// ✅ Valid Task 1: Practice Cricket  
let task1: TodoItem = {  
  title: "Practice Cricket",  
  priority: 2,  
  isDone: false  
};  
  
// ✅ Valid Task 2: Revise Maths  
let task2: TodoItem = {  
  title: "Revise Maths Chapter 3",  
  priority: 1,  
  isDone: true  
};  
  
// ❌ Invalid Task 3: Spot the Bug!  
let task3: TodoItem = {  
  title: 3, // ⚠️ Error: Title must be a string, not a number!  
  priority: 3,  
  isDone: false  
};
```

Lesson Summary & Takeaways:

- TypeScript adds static typing to JavaScript to catch bugs early.
- Core primitive types include string, number, and boolean.
- Use interface to structure complex objects like To-Do items.

Practice Questions & Assessment

Test your knowledge on TypeScript basics, types, interfaces, and debugging!

Question 1: Type Identification

Which of the following values represents a `string` data type in TypeScript?

- A) `100`
- B) `'Buy Amul Milk'`
- C) `true`
- D) `[1, 2, 3]`

Question 2: Type Compatibility

Can you assign a value of `false` to a variable declared with type `number` (e.g., `let score: number = false;`)? Explain why or why not.

Question 3: Concept Check

What is the main role of an `interface` in TypeScript?

- A) To run the code faster in the browser
- B) To act as a blueprint that defines the structure and property types of an object
- C) To change the color styling of the application user interface
- D) To store array values automatically

Question 4: Spot the Bug

Examine the TypeScript code snippet below. Identify the line with the bug and explain how to fix it:

```
interface StudentProfile {  
  name: string;  
  age: number;  
  isEnrolled: boolean;  
}  
  
let student1: StudentProfile = {  
  name: "Rahul",  
  age: "15",  
  isEnrolled: true  
}
```

```
};
```

Question 5: Practical Coding Assignment

Write an interface named `PlayerCard` for an IPL Cricket Player that contains the following properties:

- `playerName` (text)
- `jerseyNumber` (number)
- `isCaptain` (true/false)

Then, write a valid variable instance `player1` adhering to your interface.